

Experiment No: 3

Aim:- Implement Greedy Search Algorithm for Prim's Minimal Spanning Tree Algorithm.

Operating System recommended:- 64-bit Windows OS and Linux

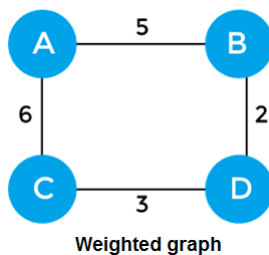
Theory:-

Minimum Spanning Tree

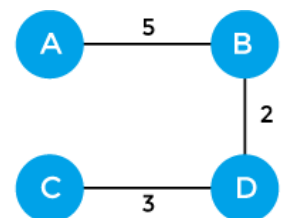
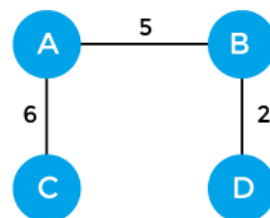
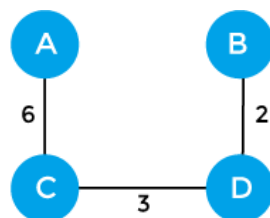
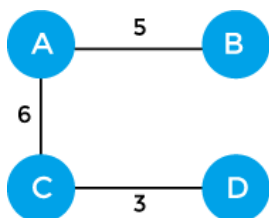
A minimum spanning tree can be defined as the spanning tree in which the sum of the weights of the edge is minimum. The weight of the spanning tree is the sum of the weights given to the edges of the spanning tree. In the real world, this weight can be considered as the distance, traffic load, congestion, or any random value.

Example of minimum spanning tree

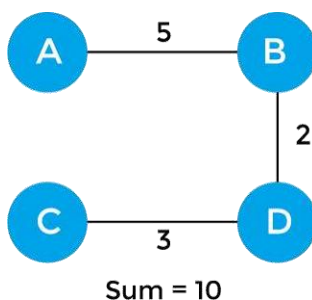
Let's understand the minimum spanning tree with the help of an example.



The sum of the edges of the above graph is 16. Now, some of the possible spanning trees created from the above graph are –



So, the minimum spanning tree that is selected from the above spanning trees for the given weighted graph is –



Applications of Minimum Spanning Tree

The applications of the minimum spanning tree are given as follows -

- Minimum spanning tree can be used to design water-supply networks, telecommunication networks, and electrical grids.
- It can be used to find paths in the map.

Algorithms for Minimum Spanning tree

A minimum spanning tree can be found from a weighted graph by using the algorithms given below -

- Prim's Algorithm
- Kruskal's Algorithm

Prim's Algorithm

It is a greedy algorithm that starts with an empty spanning tree. It is used to find the minimum spanning tree from the graph. This algorithm finds the subset of edges that includes every vertex of the graph such that the sum of the weights of the edges can be minimized.

Prim's algorithm starts with the single node and explores all the adjacent nodes with all the connecting edges at every step. The edges with the minimal weights causing no cycles in the graph got selected.

How does the Prim's Algorithm work?

Prim's algorithm is a greedy algorithm that starts from one vertex and continue to add the edges with the smallest weight until the goal is reached. The steps to implement the prim's algorithm are given as follows –

- First, we have to initialize an MST with the randomly chosen vertex.
- Now, we have to find all the edges that connect the tree in the above step with the new vertices. From the edges found, select the minimum edge and add it to the tree.
- Repeat step 2 until the minimum spanning tree is formed.

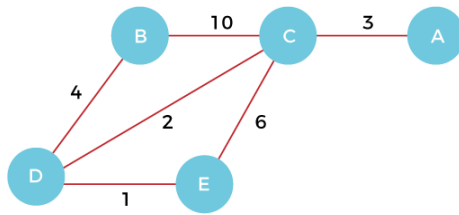
The applications of Prim's Algorithm are

- Prim's algorithm can be used in network designing.
- It can be used to make network cycles.
- It can also be used to lay down electrical wiring cables.

Example of Prim's Algorithm

Now, let's see the working of prim's algorithm using an example. It will be easier to understand the prim's algorithm using an example.

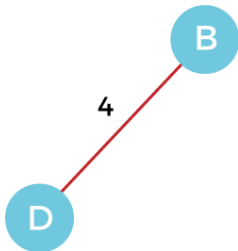
Suppose, a weighted graph is –



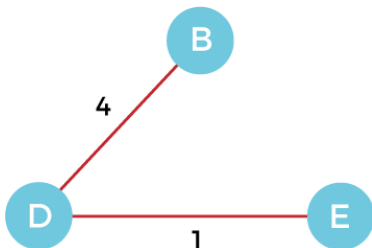
Step 1 – First, we have to choose a vertex from the above graph. Let's choose B.



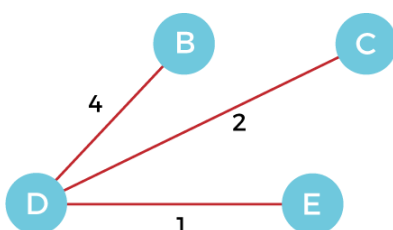
Step 2 - Now, we have to choose and add the shortest edge from vertex B. There are two edges from vertex B that are B to C with weight 10 and edge B to D with weight 4. Among the edges, the edge BD has the minimum weight. So, add it to the MST.



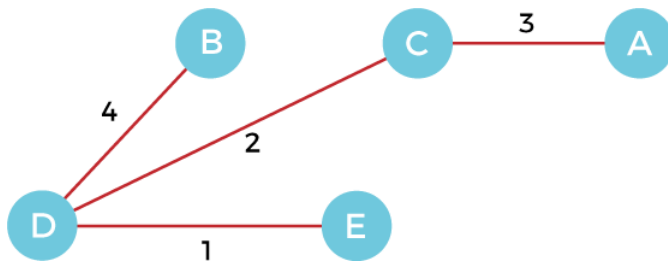
Step 3 - Now, again, choose the edge with the minimum weight among all the other edges. In this case, the edges DE and CD are such edges. Add them to MST and explore the adjacent of C, i.e., E and A. So, select the edge DE and add it to the MST.



Step 4 - Now, select the edge CD, and add it to the MST.



Step 5 - Now, choose the edge CA. Here, we cannot select the edge CE as it would create a cycle to the graph. So, choose the edge CA and add it to the MST.



So, the graph produced in step 5 is the minimum spanning tree of the given graph. The cost of the MST is given below -

Cost of MST = 4 + 2 + 1 + 3 = 10 units.

Algorithm

Step 1: Select a starting vertex

Step 2: Repeat Steps 3 and 4 until there are fringe vertices

Step 3: Select an edge 'e' connecting the tree vertex and fringe vertex that has minimum weight

Step 4: Add the selected edge and the vertex to the minimum spanning tree T

[END OF LOOP]

Step 5: EXIT

Complexity of Prim's Algorithm

Now, let's see the time complexity of Prim's algorithm. The running time of the prim's algorithm depends upon using the data structure for the graph and the ordering of edges. Below table shows some choice.

Data structure used for the minimum edge weight	Time Complexity
Adjacency matrix, linear searching	$O(V ^2)$
Adjacency list and binary heap	$O(E \log V)$
Adjacency list and Fibonacci heap	$O(E + V \log V)$

Conclusion:-

Prim's algorithm can be simply implemented by using the adjacency matrix or adjacency list graph representation, and to add the edge with the minimum weight requires the linearly searching of an array of weights. It requires $O(|V|^2)$ running time. It can be improved further by using the implementation of heap to find the minimum weight edges in the inner loop of the algorithm.

Practical No. 3

Implement Greedy search algorithm for any of the following application: • Selection Sort • Minimum Spanning Tree • Single-Source Shortest Path Problem • Job Scheduling Problem • Prim's Minimal Spanning Tree Algorithm • Kruskal's Minimal Spanning Tree Algorithm • Dijkstra's Minimal Spanning Tree Algorithm.

Program :

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
        print(f"Step {i+1}: {arr}")
    return arr

arr = [89, 45, 12, 67, 34]
print("Original array:", arr)
sorted_arr = selection_sort(arr)
print("Sorted array:", sorted_arr)
```

Output :

```
Original array: [89, 45, 12, 67, 34]
Step 1: [12, 45, 89, 67, 34]
Step 2: [12, 34, 89, 67, 45]
Step 3: [12, 34, 45, 67, 89]
Step 4: [12, 34, 45, 67, 89]
Step 5: [12, 34, 45, 67, 89]
Sorted array: [12, 34, 45, 67, 89]
```